Double Key Encryption (DKE) Implementation — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Double Key Encryption (DKE) Implementation, Microsoft Purview 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Scope boundary . . . 4

Outcome . . . 5

1. Solution Overview — Double Key Encryption (DKE) Implementation, Microsoft Purview 5

What Double Key Encryption is . . . 5

How it works . . . 5

What you control . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 7

2. Requirements Guide — Double Key Encryption (DKE) Implementation, Microsoft Purview 7

2.1 Microsoft 365 / Purview licensing for DKE . . . 7

2.2 Roles for labels, the app registration, and the group . . . 7

2.3 A host you control for the DKE key service . . . 7

2.4 .NET build/deploy toolchain for the DKE service . . . 8

2.5 Supported client platform for applying DKE labels . . . 8

2.6 A named audience for DKE (not everyone) . . . 8

2.7 Data owners, compliance & legal determination . . . 8

2.8 Change control and a documented rollback . . . 8

Checklist . . . 9

Global Requirements — Microsoft Purview Solutions 9

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 9

2. Purview roles (least-privilege) and portal access . . . 10

3. Security & Compliance PowerShell + Graph toolchain . . . 10

4. Data sources & scope in scope . . . 10

5. Compliance, records, privacy & legal stakeholders . . . 10

6. A pilot scope and change control . . . 11

3. Implementation Guide — Double Key Encryption (DKE) Implementation, Microsoft Purview 11

Step 1 — Confirm scope and the determination (Assess) . . . 12

Step 2 — Provision the host and the scope groups . . . 12

Step 3 — Obtain and configure the DKE key service . . . 12

Step 4 — Register the DKE service in Microsoft Entra ID . . . 13

Step 5 — Build, deploy, and validate the key service . . . 13
Step 6 — Create the DKE sensitivity label (Configure) . . . 13
Step 7 — Publish the label to the pilot audience (Pilot) . . . 14
Step 8 — Validate the pilot (Validate) . . . 14
Step 9 — Expand under change control (Expand) . . . 14
5. Testing Guide — Double Key Encryption (DKE) Implementation, Microsoft Purview 15
Test 1 — Key service endpoint is healthy . . . 15
Test 2 — DKE label is published and visible . . . 15
Test 3 — Applying the label double-encrypts (requires your key) . . . 15
Test 4 — Only authorized readers can open . . . 16
Test 5 — Microsoft cannot read DKE content (by design) . . . 16
Test 6 — Endpoint URL wired correctly on the label . . . 16
Test 7 — Usage / coverage report . . . 16
Pass/fail gate . . . 16
6. Troubleshooting Guide — Double Key Encryption (DKE) Implementation, Microsoft Purview16
A. "Use Double Key Encryption" option is missing on the label . . . 17
B. Key-service endpoint does not respond / bad public key . . . 17
C. Users cannot apply the DKE label . . . 17
D. Users cannot open DKE content . . . 18
E. DKE content not appearing in DLP / eDiscovery / Copilot . . . 18
F. Endpoint URL wrong on the label . . . 18
G. Diagnostics quick reference . . . 18
H. When to escalate . . . 19
7. Best Practices — Double Key Encryption (DKE) Implementation, Microsoft Purview 19
8. Deliverables Summary — Double Key Encryption (DKE) Implementation, Microsoft Purview20
Documentation . . . 20
PowerShell scripts . . . 20
What the buyer receives . . . 21
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — Double Key Encryption (DKE) Implementation, Microsoft Purview

White-label · Purview Solutions

For the most sensitive data an organization holds — trade secrets, deal data, regulated records, or information subject to a sovereignty requirement — standard encryption is not enough on its own, because the key that protects it lives entirely within the cloud provider. **Double Key Encryption (DKE)** in Microsoft Purview closes that gap: content is protected by **two** keys — one held by Microsoft and one held by **you**, in a key service **you** control. Both keys are required to decrypt, so no one — not Microsoft, not an attacker who compromises the tenant — can read the content without the key you keep. That control is the deliverable.

This is a complete, deploy-it-in-order package for standing up Double Key Encryption end to end: the **customer-held DKE key service**, the **Microsoft Entra ID app registration** that secures it, and the **sensitivity label** wired to your endpoint — piloted on a named group, validated, and expanded under change control. It is written for an administrator with no prior DKE experience.

What you get

- A plain-language Solution Overview of how DKE works — the two keys, the customer-held key service,

the label wiring, and the deliberate trade-offs (DKE content is invisible to cloud content services).

- A Requirements Guide with verify / implement-if-missing for every prerequisite (licensing, roles,

a host for the key service, the client platform, and stakeholders).

- An Implementation Guide of precise, ordered steps — deploy and secure the key service, register the

app in Entra ID, create and publish the DKE label, pilot, validate, then expand.

- Five PowerShell scripts: configuration discovery, safe remediation (default `-WhatIf`), coverage

validation, a DKE usage report, and a scope + key-service health helper.

- Testing and Troubleshooting guides for a confident rollout.

What this delivers

A named, reusable set of objects — a customer-held DKE key service, an Entra app registration that authorizes it, and a Double-Key-Encryption sensitivity label published to a pilot group — deployed on the assess-configure-pilot model, validated end to end (key held by you, both keys required to decrypt), with a documented rollback.

The standard deployment model

Assess → **Design** → **Configure** → **Pilot (test scope)** → **Validate** → **Expand.**

Scope boundary

Which data is "most sensitive" and must be double-keyed, and any regulatory or sovereignty determination, are **compliance, records, privacy, and legal** decisions — this package delivers the technical capability and the evidence, not the classification determination.

Outcome

The second key stays under your control in Microsoft Purview — Double Key Encryption configured for your most sensitive data, the key service you own, and the label wiring proven on a pilot before you scale.

1. Solution Overview — Double Key Encryption (DKE) Implementation, Microsoft Purview

What Double Key Encryption is

Double Key Encryption (DKE) is a Microsoft Purview Information Protection capability for the organization's **most sensitive** data. Content protected with a DKE sensitivity label is encrypted with **two** keys:

1. A **Microsoft-held key**, stored in Azure as part of Microsoft Purview / Azure Rights Management —

the same key used by ordinary sensitivity-label encryption.

1. A **customer-held key**, stored in a **key service that you deploy and control** (the DKE service),

which never leaves your control.

Both keys are required to decrypt the content. Because Microsoft never holds your key, no one can read DKE-protected content without access to your key service — not Microsoft, and not an adversary who compromises your Microsoft 365 tenant. That is the entire point: you keep control of the second key.

How it works

1. You deploy the **DKE key service** — an open-source reference web service published by Microsoft —

onto hosting **you** control (for example an Azure App Service web app or an on-premises/IIS server). The service holds your key pair and exposes a public-key endpoint plus a decrypt/wrap-unwrap endpoint over HTTPS.

1. You **register the DKE service as an application in Microsoft Entra ID** and secure its endpoints

with OAuth, so only authorized users in your tenant can request the customer-held key operations.

1. In the Microsoft Purview portal you create a **sensitivity label** whose encryption is set to

Use Double Key Encryption, and you enter your DKE **endpoint URL** on the label. You publish the label with a label policy to a pilot group.

1. A user applies the DKE label to a document or email in a supported Office client. The client

obtains the Microsoft key from Azure and the **customer key from your DKE service** (after the user authenticates); the content is double-encrypted. To open it later, the client must again reach **both** key sources.

What you control

• **Where your key lives** — in a service you deploy, on hosting you own, in a region you choose (a sovereignty and custody control).

• **Who can use the key** — through the Entra ID app registration, OAuth, and the authorized-user configuration on the DKE service.

• **Which data is double-keyed** — through the DKE sensitivity label and where you publish it (a small named audience for the most sensitive data, not everyone).

• **Availability of the second key** — because decryption depends on your service, its uptime and reachability are yours to run.

Benefits

• **Customer key custody** — the second key is yours; Microsoft cannot decrypt the content, which is exactly what the most sensitive data and many sovereignty requirements call for.

• **Defense in depth for the crown jewels** — a tenant compromise does not expose DKE content without

also compromising your separate key service.

• **Same label experience** — users apply a sensitivity label like any other; the double-key mechanics are transparent to them.

• **Defensible and evidenced** — the capability is configured, piloted, and validated, with a clear record of the key you hold.

Limitations (be honest with buyers)

• **DKE content is invisible to cloud content services — by design.** Because Microsoft cannot decrypt

it, DKE-protected content **cannot** be inspected by content-based DLP, indexed for eDiscovery content search, scanned by Microsoft 365 co-authoring/AutoSave, or read by Copilot or other cloud processing. Reserve DKE for the small set of data where that trade-off is acceptable. **Verify the current list of unsupported services for your tenant** in Microsoft's documentation.

• **Client-platform support is limited and evolving.** DKE labels are applied through built-in labeling in **Office desktop apps on Windows**; web and other platforms have historically not supported DKE. **Verify current platform/client support for your tenant** before you scope users.

• **You run the key service.** Its availability, patching, backup, and disaster recovery are your responsibility; if the service is unreachable, DKE content cannot be opened.

• **Licensing-gated.** DKE requires the appropriate Microsoft 365 / Purview plan (see the Requirements Guide) — **verify your licensing/tenant.**

The standard deployment model

> ***Deploy the customer-held key service → secure it with an Entra ID app registration → create and publish the DKE label → pilot on a named group → validate (both keys required, key held by you) → expand under change control.*** *Repeatable pattern:* ***Assess → Design → Configure → Pilot → Validate → Expand****, with a documented rollback at every stage.*

Which data is sensitive enough to warrant DKE, and any regulatory or sovereignty applicability, are deferred to the organization's compliance, records, privacy, and legal functions — this solution provides the technical configuration and the evidence, not the determination.

2. Requirements Guide — Double Key Encryption (DKE) Implementation, Microsoft Purview

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (licensing, least-privilege roles, the S&C PowerShell / Graph toolchain, in-scope data, stakeholders, and a pilot scope with change control) — this guide adds the DKE-specific prerequisites.

2.1 Microsoft 365 / Purview licensing for DKE

Description: Licensing that includes Microsoft Purview Information Protection with **Double Key Encryption** — DKE requires the advanced information-protection plan (**Microsoft 365 E5** or the **E5 Compliance / Information Protection and Governance** add-on). **Why:** DKE is a premium, plan-gated capability; the **Use Double Key Encryption** option will not appear on a sensitivity label without the required plan. **Verify:** Microsoft 365 admin center → **Billing** → **Licenses**; confirm the DKE option is available when configuring a label's encryption in the Purview portal. Run `IPDKE-Discovery.ps1`. **Verify your licensing/tenant** against Microsoft's current DKE documentation. **Implement:** Assign/purchase the required plan before starting.

2.2 Roles for labels, the app registration, and the group

Description: Least-privilege roles across the three areas this solution touches: **Compliance Administrator** or **Information Protection Admin** (create and publish the DKE label); **Application Administrator** + **Cloud Application Administrator** (register the DKE service app and grant admin consent — Global Administrator only where consent requires it); **Groups Administrator** (create the pilot/reader groups). **Why:** DKE spans Purview labeling, Entra ID app registration/consent, and group management; each needs its matching role. **Verify:** Purview portal → **Settings** → **Roles and scopes**; Entra admin center → **Roles and admins**. Confirm membership. **Implement:** Assign the least-privilege roles under change control; reserve Global Administrator for the consent step only if required.

2.3 A host you control for the DKE key service

Description: Hosting **you** own and control on which to deploy the DKE key service — for example an **Azure App Service** web app or an **on-premises / IIS** server — reachable over **HTTPS** by the clients that will open DKE content, with a valid TLS certificate and a DNS name (for example `dke.contoso.com`). **Why:** The customer-held key must live on infrastructure you control; the whole security benefit depends on

this service being separate from — and not decryptable by — Microsoft. Clients must reach it to open content. **Verify:** The host exists, resolves in DNS, and serves HTTPS with a trusted certificate; the network path from client machines is open. **Implement:** Provision the host and certificate; decide the hosting region to satisfy any sovereignty requirement (a compliance/legal decision — see Requirement 2.7).

2.4 .NET build/deploy toolchain for the DKE service

Description: A workstation with the **.NET SDK** and **Git** to obtain, configure, and build the Microsoft-published open-source DKE key service, plus the means to publish it to the host in 2.3. **Why:** The DKE service is deployed from Microsoft's open-source reference implementation, which you configure (appsettings) and build before publishing. **Verify:** `dotnet --info` and `git --version` succeed on the build workstation. **Implement:** Install the current .NET SDK and Git; obtain the Microsoft DKE service source. **Follow Microsoft's current DKE deployment documentation for exact versions and steps.**

2.5 Supported client platform for applying DKE labels

Description: The users who will apply/open DKE content run a **supported client** — historically **Microsoft 365 Apps (Office) desktop on Windows** with built-in labeling. **Verify current platform/client support for your tenant** (web, macOS, and mobile support has been limited and is evolving). **Why:** DKE labels can only be applied and opened on supported clients; scoping the pilot to unsupported clients will fail. **Verify:** Confirm pilot users run a supported Office desktop build on Windows with built-in labeling; check Microsoft's current DKE client-support list. **Implement:** Provision supported clients for the pilot group; do not scope DKE to unsupported platforms.

2.6 A named audience for DKE (not everyone)

Description: A **small, named audience** — a Microsoft Entra security group of pilot users (`Purview - DKE Pilot Users`) and the set of people authorized to read the double-keyed content (`SG - DKE Ultra Confidential Readers`) — reflecting that DKE is for the **most sensitive** data, not the whole organization. **Why:** DKE trades away cloud content services (DLP inspection, eDiscovery content search, Copilot); applying it broadly would break routine workflows. It belongs on a narrow, high-value scope. **Verify:** Entra → **Groups**; the pilot and reader groups exist. Run `IPDKE-Scope-And-ServiceHealth.ps1`. **Implement:** Create the groups (the helper script can create the pilot group) and agree the audience with data owners.

2.7 Data owners, compliance & legal determination

Description: Named **data owners** and engagement with **compliance, records, privacy, and legal** stakeholders who decide **which** data must be double-keyed and confirm any **sovereignty / regulatory** driver (including the required hosting region for the key service). **Why:** "Which data is sensitive enough for DKE" and any regulatory/sovereignty applicability are business and legal determinations, not IT defaults; the hosting region may be legally constrained. **Verify:** A named owner and a documented determination exist for the data to be double-keyed and the key-service region. **Implement:** Identify owners; capture the determination and the region requirement before configuring (see the vendor boundary — determinations are deferred to these functions).

2.8 Change control and a documented rollback

Description: A documented **change-control / rollback** process for publishing the DKE label and for the key-service configuration. **Why:** DKE changes how sensitive content is protected and depends on a service you run; a pilot gate and a rollback are mandatory. **Verify:** A change ticket exists with an approver, the pilot scope, and a rollback (unpublish the label policy; the label stays but is applied to no one). **Implement:** Record the change, name the pilot scope, define the rollback, and get approval before publishing beyond the pilot.

Checklist

- Microsoft 365 / Purview licensing includes DKE (E5 / E5 Compliance) — `IPDKE-Discovery.ps1`
- Roles: Compliance Administrator / Information Protection Admin; Application + Cloud Application Administrator; Groups Administrator
- A host you control for the DKE key service (HTTPS, TLS cert, DNS, client reachability)
- .NET SDK + Git build/deploy toolchain for the DKE service
- Supported client platform for pilot users (verify current DKE client support)
- Named audience groups created — `Purview - DKE Pilot Users`, `SG - DKE Ultra Confidential Readers` — `IPDKE-Scope-And-ServiceHealth.ps1`
- Data owners named; compliance/legal determination + hosting-region requirement captured
- Change control + documented rollback (unpublish the label policy)

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.
- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.
- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.
- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.
- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Double Key Encryption (DKE) Implementation, Microsoft Purview

Follow these steps **in order**. DKE spans three surfaces: the **customer-held key service** (hosting you own), **Microsoft Entra ID** (`https://entra.microsoft.com`), and the **Microsoft Purview portal** (`https://purview.microsoft.com`). Where exact versions, file names, or endpoint paths depend on Microsoft's evolving reference implementation, **follow Microsoft's current Double Key Encryption deployment documentation** — do not assume; the named objects and the order below are stable.

> ***Model:*** *Deploy and secure your key service, wire it to a sensitivity label, then pilot the label on a small named audience before you expand. Pattern: Assess → Design → Configure → Pilot → Validate → Expand, with a documented rollback (unpublish the label policy).*

The named objects created below are reused verbatim in Testing, Troubleshooting, and the scripts:

Object	Name
DKE key service host (example)	`dke.contoso.com`
DKE key name	`DKE-Key-01`
DKE endpoint URL (public key)	`https://dke.contoso.com/DKE-Key-01`
Entra app registration	`App - DKE Key Service`
Sensitivity label	`Sensitivity Label - Ultra Confidential (DKE) - Named Group`
Label policy	`Label Policy - DKE Ultra Confidential - DKE Pilot`
Pilot users group	`Purview - DKE Pilot Users`
Authorized readers group	`SG - DKE Ultra Confidential Readers`

Step 1 — Confirm scope and the determination (Assess)

1. With data owners and compliance/legal, confirm **which** data is sensitive enough for DKE and the required **hosting region** for the key service (Requirement 2.7). DKE is for the **most sensitive** data only.

1. Confirm the trade-off is accepted: DKE content will **not** be inspected by content DLP, indexed for eDiscovery content search, or processed by Copilot/co-authoring (Overview, Limitations).

1. Record the decision. This is the input to the label's audience in Step 6.

Step 2 — Provision the host and the scope groups

1. Provision the HTTPS host you control (Requirement 2.3) — for example `dke.contoso.com` with a trusted TLS certificate and DNS — in the region agreed in Step 1.

1. Create the audience groups with `IPDKE-Scope-And-ServiceHealth.ps1`:

• `Purview - DKE Pilot Users` (who will pilot applying the label),

• `SG - DKE Ultra Confidential Readers` (who is authorized to read DKE content).

Step 3 — Obtain and configure the DKE key service

1. On the build workstation (Requirement 2.4), obtain **Microsoft's open-source Double Key Encryption service** source (published by Microsoft on GitHub) using `git`.

1. Edit the service's `appsettings.json` per Microsoft's documentation. The key settings are:

• `AuthorizedEmailAddress` — the users/domains allowed to request key operations (align to `SG - DKE Ultra Confidential Readers`).

• `JwtAudience` — the app ID/URI of `App - DKE Key Service` (from Step 4).

• `JwtIssuer` — your tenant's token issuer, `https://sts.windows.net/<your-tenant-id>/`.

• `TestKeys` **/ key name and ID** — set the key **Name** to `DKE-Key-01` and a generated **GUID**.

• **Issuer/Audience validation** — leave validation **on**.

1. Generate the DKE **key pair** as Microsoft's documentation directs (this is the customer-held key — keep the private key under your control; **never** commit it to source control or share it).

Step 4 — Register the DKE service in Microsoft Entra ID

1. Entra admin center → **Applications** → **App registrations** → **New registration**. Name it `App - DKE Key Service`. Set the redirect/URI values per Microsoft's DKE documentation.

1. **Expose an API:** set the Application ID URI and add the delegated scope Microsoft's DKE docs specify (for example `user_impersonation`).

1. **API permissions / consent:** grant the required permission and **grant admin consent** for the tenant (Application/Cloud Application Administrator; Global Administrator only if consent requires it).

1. Copy the **Application (client) ID** and Application ID URI into the service's `appsettings.json` (`JwtAudience`, Step 3) so the two match.

Step 5 — Build, deploy, and validate the key service

1. Build the service with the .NET SDK and **publish** it to the host from Step 2 (Azure App Service or IIS), served over **HTTPS**.

1. **Validate the deployment (read-only):** browse to the public-key endpoint `https://dke.contoso.com/DKE-Key-01`. It must return the **public key JSON** for `DKE-Key-01` over a trusted TLS connection. Run `IPDKE-Scope-And-ServiceHealth.ps1 -DkeEndpointUrl https://dke.contoso.com/DKE-Key-01` to probe reachability, TLS, and the JSON shape.

1. **Pass gate:** the endpoint returns the public key over HTTPS and is reachable from a pilot client. Do not proceed until this passes.

Step 6 — Create the DKE sensitivity label (Configure)

1. Purview portal → **Information Protection** → **Sensitivity labels** → **Create a label**.

2. **Name and tooltip:** enter `Sensitivity Label - Ultra Confidential (DKE) - Named Group` and a clear user-facing tooltip explaining this is for the most sensitive data.

1. **Scope:** select **Items** (files and emails). **Next**.

2. **Encryption:** choose **Configure encryption**, then select **Use Double Key Encryption** and enter your **DKE endpoint URL** `https://dke.contoso.com/DKE-Key-01`. (If **Use Double Key Encryption** does not appear, your licensing does not include DKE — Requirement 2.1.)

1. **Permissions:** assign permissions to `SG - DKE Ultra Confidential Readers` (for example

Co-Author or **Viewer** as the business requires). **Next**.

1. **Content marking:** optionally add a header/footer/watermark such as `ULTRA CONFIDENTIAL — DKE`.

Next.

1. Leave auto-labeling **off** — DKE is applied deliberately by users on the most sensitive items, not automatically. **Next** → **Create the label.**

Step 7 — Publish the label to the pilot audience (Pilot)

1. Purview portal → **Information Protection** → **Sensitivity labels** → **Label policies** → **Publish labels**.
2. **Choose labels to publish:** add `Sensitivity Label - Ultra Confidential (DKE) - Named Group`.

Next.

1. **Assign to users and groups:** add **only** `Purview - DKE Pilot Users` for the pilot. **Next**.
2. **Policy settings:** leave defaults; do **not** set DKE as a default or mandatory label. **Next**.
3. **Name:** `Label Policy - DKE Ultra Confidential - DKE Pilot`. **Next** → **Submit.** Allow time (up to 24 hours) for the label to appear in pilot users' Office clients.

Step 8 — Validate the pilot (Validate)

1. Complete the Testing Guide against `Purview - DKE Pilot Users` on a supported Windows Office desktop client.
1. Confirm the core proofs: the DKE label appears; applying it double-encrypts the item; opening it requires reaching **your** key service; and a user **outside** `SG - DKE Ultra Confidential Readers` is denied.
1. Run `IPDKE-Validation.ps1` (label present with DKE encryption and the expected endpoint URL, label policy published to the pilot group, endpoint reachable) and `IPDKE-Report.ps1` (DKE label usage / coverage). Confirm the endpoint health with `IPDKE-Scope-And-ServiceHealth.ps1`.
1. **Gate:** proceed to Expand only when every proof passes and the key service is healthy and monitored.

Step 9 — Expand under change control (Expand)

1. Once the pilot passes, widen the label policy `Label Policy - DKE Ultra Confidential - DKE Pilot`

from `Purview - DKE Pilot Users` to the broader approved audience (still a **named** audience for the most sensitive data — not All users).

1. Add authorized readers to `SG - DKE Ultra Confidential Readers` as the business approves.
2. Put the **key service into production operations** — monitoring, backup of the customer key, patching, and a documented disaster-recovery plan (its availability now gates access to DKE content).
1. Re-validate each expansion with `IPDKE-Validation.ps1`, `IPDKE-Report.ps1`, and `IPDKE-Scope-And-ServiceHealth.ps1`.
1. **Rollback (if needed):** unpublish the label policy (Purview → **Label policies** → delete/disable `Label Policy - DKE Ultra Confidential - DKE Pilot`). The label remains defined but is applied to no one; **already-protected content still requires both keys to open**, so keep the key service running during and after any rollback.

Order of operations

`Assess/determine → host + groups → obtain & configure key service → Entra app registration → build/deploy & validate endpoint → create DKE label (endpoint URL) → publish to pilot → validate → expand`

5. Testing Guide — Double Key Encryption (DKE) Implementation, Microsoft Purview

Run after the key service is deployed (Implementation Step 5) and the label is published to `Purview - DKE Pilot Users` (Step 7), and again after each expansion. Apply/open DKE content on a **supported Windows Office desktop client** (Requirement 2.5).

Test 1 — Key service endpoint is healthy

Do: Browse to `https://dke.contoso.com/DKE-Key-01`, or run `IPDKE-Scope-And-ServiceHealth.ps1 -DkeEndpointUrl https://dke.contoso.com/DKE-Key-01`. **Expected:** The endpoint returns the **public key JSON** for `DKE-Key-01` over a trusted HTTPS/TLS connection and is reachable from a pilot client.

Test 2 — DKE label is published and visible

Do: As a member of `Purview - DKE Pilot Users`, open the **Sensitivity** button in Word or Outlook (desktop, Windows). Or run `IPDKE-Validation.ps1`. **Expected:** `Sensitivity Label - Ultra Confidential (DKE) - Named Group` appears via `Label Policy - DKE Ultra Confidential - DKE Pilot`.

Test 3 — Applying the label double-encrypts (requires your key)

Do: Apply the DKE label to a test document and save. Then, on a client that **cannot reach** `dke.contoso.com` (block it or disconnect from the key service network path), try to open the file.

Expected: Applying the label succeeds only after the client authenticates and reaches **your** DKE service; when the service is unreachable the content **cannot** be opened — proving the customer-held key is required.

Test 4 — Only authorized readers can open

Do: As a user **in** `SG - DKE Ultra Confidential Readers`, open the DKE-protected item. Then, as a user **outside** that group, attempt to open the same item. **Expected:** The authorized reader opens it; the unauthorized user is **denied** per the label's permissions — even inside the tenant.

Test 5 — Microsoft cannot read DKE content (by design)

Do: Confirm DKE content is excluded from cloud content services: check that a content-based DLP policy does not match the DKE item's content, and that eDiscovery **content search** does not index its body. **Verify the current unsupported-services list for your tenant. Expected:** DKE content is not readable by cloud content services — the intended trade-off; metadata (not content) may still be visible.

Test 6 — Endpoint URL wired correctly on the label

Do: Run `IPDKE-Validation.ps1`. **Expected:** The label reports DKE encryption enabled and the DKE endpoint URL matches `https://dke.contoso.com/DKE-Key-01`; the label policy targets `Purview - DKE Pilot Users`.

Test 7 — Usage / coverage report

Do: Run `IPDKE-Report.ps1`. **Expected:** A summary of the DKE label(s), the publishing policy and its audience, and a reminder of the content-service exclusions — so coverage and scope are visible.

Pass/fail gate

Gate	Requirement
Service pass	Test 1 passes — the endpoint returns the public key over HTTPS and is reachable
Pilot pass	Tests 2–4, 6 pass on `Purview - DKE Pilot Users`; the customer key is provably required (Test 3) and permissions hold (Test 4)
Trade-off confirmed	Test 5 confirms the accepted content-service exclusions
Expand pass	Tests 1, 2, 4, 6, 7 pass after widening the audience; the key service is monitored and backed up

6. Troubleshooting Guide — Double Key Encryption (DKE) Implementation, Microsoft Purview

Use `IPDKE-Discovery.ps1` for the tenant's current label/DKE config, `IPDKE-Validation.ps1` to confirm the named objects and endpoint, `IPDKE-Report.ps1` for DKE usage and the content-service exclusions, `IPDKE-Scope-And-ServiceHealth.ps1` to probe the key service and build scope groups, and `IPDKE-Remediation.ps1` (default `-WhatIf`) for the one safe fix (enable a disabled label policy).

A. "Use Double Key Encryption" option is missing on the label

Symptom	Cause	Remediation
No DKE option when configuring a label's encryption	Licensing does not include DKE	Confirm Microsoft 365 E5 / E5 Compliance (Requirement 2.1); run `IPDKE-Discovery.ps1`; **verify your licensing/tenant** against Microsoft's current DKE docs
Option present but greyed out	Role lacks label-configuration rights	Confirm Compliance Administrator / Information Protection Admin (Requirement 2.2)

B. Key-service endpoint does not respond / bad public key

• Browse to `https://dke.contoso.com/DKE-Key-01`; run

`IPDKE-Scope-And-ServiceHealth.ps1 -DkeEndpointUrl https://dke.contoso.com/DKE-Key-01`.

• If unreachable: check the host is running, DNS resolves, the firewall/network path is open, and the service is bound to HTTPS. If TLS fails: confirm a trusted certificate (not self-signed) for the DNS name.

• If it responds but not with the public-key JSON: re-check the service's `appsettings.json` key **Name** (`DKE-Key-01`) and that the key pair was generated per Microsoft's docs; the URL path must match the configured key name.

C. Users cannot apply the DKE label

Symptom	Cause	Remediation
DKE label not visible in Office	Label policy not published to the user, or still propagating	Confirm the user is in `Purview - DKE Pilot Users` and that `Label Policy - DKE Ultra Confidential - DKE Pilot` targets them; allow up to 24 hours; `IPDKE-Validation.ps1`
Label visible but applying fails	Unsupported client/platform	Confirm a supported Windows Office desktop build with built-in labeling (Requirement 2.5); **verify current client support**

Symptom	Cause	Remediation
Apply fails with an auth/consent error	Entra app registration or admin consent incomplete	Re-check `App - DKE Key Service` (Application ID URI, exposed scope, admin consent granted); confirm `JwtAudience`/`JwtIssuer` in `appsettings.json` match the app and tenant

D. Users cannot open DKE content

Symptom	Cause	Remediation
Authorized user cannot open the item	Client cannot reach the key service	Confirm reachability of `https://dke.contoso.com/DKE-Key-01` from the client network; the key service must be up (its availability gates access)
User denied opening the item	Not in the authorized readers	Confirm the user is in `SG - DKE Ultra Confidential Readers` and in the service's authorized-users setting; the two must align
Opens on one machine, not another	Endpoint blocked on that network segment	Open the network path to the DKE endpoint for that segment

E. DKE content not appearing in DLP / eDiscovery / Copilot

- **This is by design, not a fault.** Microsoft cannot decrypt DKE content, so it is excluded from

content-based DLP, eDiscovery content search, co-authoring, and Copilot. Confirm the accepted trade-off (Overview, Limitations; Testing Test 5) and **verify the current unsupported-services list for your tenant**. If content services are required for this data, it should not be DKE-protected.

F. Endpoint URL wrong on the label

- Run `IPDKE-Validation.ps1 -DkeEndpointUrl https://dke.contoso.com/DKE-Key-01`; if the label's URL

does not match, edit the label in the Purview portal (Information Protection → Sensitivity labels → the label → **Encryption**) and correct the DKE endpoint URL. Changing it here is a portal decision — `IPDKE-Remediation.ps1` deliberately never edits DKE encryption.

G. Diagnostics quick reference

```
# Security & Compliance PowerShell (Exchange Online Management module)
Connect-IPPSSession
Get-Label | Select-Object DisplayName, Priority, EncryptionEnabled
Get-LabelPolicy | Select-Object Name, Enabled
# DKE endpoint (read-only reachability)
```

```
Invoke-WebRequest -Uri 'https://dke.contoso.com/DKE-Key-01' -UseBasicParsing | Select-Object StatusCode
```

Property names for the DKE endpoint URL on a label can vary by module version — the scripts probe the likely names; verify against Microsoft's current DKE documentation.

H. When to escalate

If licensing is correct, the label policy is published and propagated, the user is a supported client in scope, the app registration/consent is complete, and the key-service endpoint is healthy — yet DKE still fails after 24 hours — collect the output of `IPDKE-Discovery.ps1`, `IPDKE-Validation.ps1`, and `IPDKE-Scope-And-ServiceHealth.ps1`, plus the key-service logs, and open a Microsoft support case. Infrastructure issues with the key service (hosting, TLS, network) are handled by the team that owns the host; regulatory/sovereignty questions go to compliance/legal.

7. Best Practices — Double Key Encryption (DKE) Implementation, Microsoft Purview

- **Reserve DKE for the crown jewels.** DKE deliberately removes cloud content services (content DLP, eDiscovery content search, Copilot, co-authoring). Apply it to a **small, named** set of the most sensitive data — never as a broad default — so you do not break routine workflows.
- **Treat the key service as production infrastructure.** The customer key's availability gates access to all DKE content. Run it with monitoring, redundancy, patching, and a tested disaster-recovery plan; alert on the endpoint before users notice they cannot open files. Probe it with `IPDKE-Scope-And-ServiceHealth.ps1`.
- **Protect the customer key like the secret it is.** Back it up securely, restrict who can touch the host, and **never** commit the private key or `appsettings.json` secrets to source control or share them. Losing the key means losing access to the content it protects.
- **Choose the hosting region deliberately.** If DKE is driven by a sovereignty requirement, host the key service in the region compliance/legal require, and record the decision — the region is part of the control.
- **Validate the endpoint before you wire the label.** Confirm `https://dke.contoso.com/DKE-Key-01` returns the public key over trusted HTTPS (Implementation Step 5) before entering it on the label; a bad endpoint URL is the most common DKE failure.
- **Pilot on a named group first.** Publish `Label Policy - DKE Ultra Confidential - DKE Pilot` to `Purview - DKE Pilot Users` and prove the mechanics — both keys required, permissions enforced — before widening.
- **Keep the audience and the readers explicit.** Publish the label only to a named audience, and keep `SG - DKE Ultra Confidential Readers` (and the service's authorized-users setting) tight and in sync; DKE permissions and key-access authorization are two separate controls — align them.

• **Name every object by purpose and stage.** Use the `<Purpose> - <Scope> - <Stage>` convention (for

example `... - DKE Pilot`) so the portal shows exactly where the label and policy are in their lifecycle.

• **Keep the key service running through any rollback.** Unpublishing the label stops new labeling, but

content already protected still needs **both** keys to open — never decommission the key service while DKE content exists.

• **Document the objects and the rollback.** Record the endpoint URL, key name, app registration, label

and policy names, audience, hosting region, and the rollback (unpublish the label policy) for repeatability, change control, and audit evidence.

• **Verify platform and service support against current Microsoft docs.** DKE client-platform support

and the unsupported-cloud-services list evolve — re-check them for your tenant before expanding.

8. Deliverables Summary — Double Key Encryption (DKE) Implementation, Microsoft Purview

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What DKE is — the two keys, the customer-held key service, the label wiring — benefits and the deliberate trade-offs
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (licensing, roles, host, .NET toolchain, client platform, audience, determination)
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — deploy & secure the key service, register the Entra app, create & publish the DKE label, pilot, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates (key required, permissions, content-service exclusion)
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`IPDKE-Discovery.ps1`	Reports the tenant's sensitivity labels, flags which use encryption / Double Key Encryption and their endpoint URLs, lists label policies, and shows a DKE licensing signal (read-only)
`IPDKE-Validation.ps1`	Confirms the DKE label exists with DKE encryption and the expected endpoint URL, the label policy is published to the pilot group, and the endpoint is reachable (read-only)
`IPDKE-Report.ps1`	DKE label usage / coverage summary — DKE labels, their publishing policies and audiences, plus the content-service exclusions (read-only)
`IPDKE-Remediation.ps1`	Safe remediation of common gaps (default `-WhatIf`) — enables a published-but-disabled label policy; never creates or alters DKE encryption on a label (a portal decision)
`IPDKE-Scope-And-ServiceHealth.ps1`	Scope + health helper — creates/updates the pilot & reader Entra groups (gated by `-Apply`) and probes the DKE endpoint's reachability, TLS, and public-key JSON (read-only probe)

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the model: **Assess → Design → Configure → Pilot → Validate → Expand**, with endpoint and pilot gates and a documented rollback.

Outcome

The second key stays under your control in Microsoft Purview — a customer-held DKE key service, an Entra app registration that secures it, and a Double-Key-Encryption sensitivity label wired to your endpoint and proven on a pilot before you scale. Which data warrants DKE, and any sovereignty determination, remain with compliance, records, privacy, and legal.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IPDKE-Discovery.ps1

Purpose: Connects to Security & Compliance PowerShell and reports the tenant's sensitivity labels, highlighting which ones use encryption and, specifically, which use Double Key Encryption (with the configured DKE endpoint URL), plus the label policies that publish them. This is the "what exists today" read used to plan or validate a DKE rollout. It changes nothing. HOW TO RUN Install-Module ExchangeOnlineManagement -Scope CurrentUser powershell -ExecutionPolicy Bypass -File .\IPDKE-Discovery.ps1 Sign in as a Compliance Administrator (or Information Protection Admin) when prompted. EXPECTED RESULTS A list of sensitivity labels (name, priority, encryption on/off), a DKE section listing any labels whose encryption is Double Key Encryption and their endpoint URLs, and the label policies (name, enabled). If no DKE label exists yet, that is reported as a normal starting state. .NOTES White-label — Purview Solutions. Read-only. No secrets. Does not modify any label or policy. Uses Get-Label / Get-LabelPolicy. The DKE endpoint URL is read from the label's DKE property when present; property names can vary by tenant/module version — verify against Microsoft's current docs. > [CmdletBinding()] param()

How to run: Install-Module ExchangeOnlineManagement -Scope CurrentUser powershell -ExecutionPolicy Bypass -File .\IPDKE-Discovery.ps1 Sign in as a Compliance Administrator (or Information Protection Admin) when prompted. EXPECTED RESULTS A list of sensitivity labels (name, priority, encryption on/off), a DKE section listing any labels whose encryption is Double Key Encryption and their endpoint URLs, and the label policies (name, enabled). If no DKE label exists yet, that is reported as a normal starting state. .NOTES White-label — Purview Solutions. Read-only. No secrets. Does not modify any label or policy. Uses Get-Label / Get-LabelPolicy. The DKE endpoint URL is read from the label's DKE property when present; property names can vary by tenant/module version — verify against Microsoft's current docs. > [CmdletBinding()] param()

IPDKE-Remediation.ps1

Purpose: Remediates only the safe, low-risk gap in a DKE rollout: enabling a DKE label policy that exists but is disabled. It NEVER creates or alters Double Key Encryption on a label, never changes the endpoint URL, and never touches the key service or the Entra app registration — those are deliberate portal / infrastructure decisions. Runs in -WhatIf by default; pass -Apply to make the single safe change. HOW TO RUN Preview (safe, default): powershell -ExecutionPolicy Bypass -File .\IPDKE-Remediation.ps1 Apply: powershell -ExecutionPolicy Bypass -File .\IPDKE-Remediation.ps1 -Apply Sign in as a Compliance Administrator (or Information Protection Admin) when prompted. EXPECTED RESULTS Reports whether the DKE label policy exists and is enabled; in -Apply it enables a published-but-disabled policy. If the label or policy is missing, it points to the Implementation Guide step to create it (never auto-creates DKE encryption). .NOTES White-label — Purview Solutions. Default is a dry run. No secrets. Never enables/edits DKE encryption on a label. Uses Get-/Set-LabelPolicy and reads Get-Label. > [CmdletBinding()] param([switch]$Apply, [string]$LabelName = 'Sensitivity Label - Ultra Confidential (DKE) - Named Group', [string]$PolicyName = 'Label Policy - DKE Ultra Confidential - DKE Pilot') $WhatIf = -not $Apply $mode = $(if($WhatIf){'WHATIF (preview only)'}else{'APPLY'})

How to run: Preview (safe, default): powershell -ExecutionPolicy Bypass -File .\IPDKE-Remediation.ps1 Apply: powershell -ExecutionPolicy Bypass -File .\IPDKE-Remediation.ps1 -Apply Sign in as a Compliance

Administrator (or Information Protection Admin) when prompted. EXPECTED RESULTS Reports whether the DKE label policy exists and is enabled; in -Apply it enables a published-but-disabled policy. If the label or policy is missing, it points to the Implementation Guide step to create it (never auto-creates DKE encryption). .NOTES White-label — Purview Solutions. Default is a dry run. No secrets. Never enables/edits DKE encryption on a label. Uses Get-/Set-LabelPolicy and reads Get-Label. > [CmdletBinding()] param([switch]$Apply, [string]$LabelName = 'Sensitivity Label - Ultra Confidential (DKE) - Named Group', [string]$PolicyName = 'Label Policy - DKE Ultra Confidential - DKE Pilot') $WhatIf = -not $Apply $mode = $(if($WhatIf){'WHATIF (preview only)'}else{'APPLY'})

IPDKE-Report.ps1

Purpose: Produces a coverage summary for the DKE rollout: the sensitivity labels that use Double Key Encryption (and their endpoint URLs), the label policies that publish them and their audience, and a standing reminder of the cloud content-services that cannot process DKE content. Use it at the pilot gate and after each expansion to see exactly where DKE is applied. Read-only. HOW TO RUN Install-Module ExchangeOnlineManagement -Scope CurrentUser powershell -ExecutionPolicy Bypass -File .\IPDKE-Report.ps1 Optional: -CsvPath .\dke-report.csv to also write the label rows to CSV. Sign in as a Compliance Administrator (or Information Protection Admin) when prompted. EXPECTED RESULTS A table of DKE labels (name, endpoint URL), a table of label policies (name, enabled), and a note listing the content-service exclusions to confirm the accepted trade-off. .NOTES White-label — Purview Solutions. Read-only. No secrets. No changes to any object. Uses Get-Label / Get-LabelPolicy. Verify the current unsupported-services list in Microsoft docs. > [CmdletBinding()] param([string]$CsvPath = '')

How to run: Install-Module ExchangeOnlineManagement -Scope CurrentUser powershell -ExecutionPolicy Bypass -File .\IPDKE-Report.ps1 Optional: -CsvPath .\dke-report.csv to also write the label rows to CSV. Sign in as a Compliance Administrator (or Information Protection Admin) when prompted. EXPECTED RESULTS A table of DKE labels (name, endpoint URL), a table of label policies (name, enabled), and a note listing the content-service exclusions to confirm the accepted trade-off. .NOTES White-label — Purview Solutions. Read-only. No secrets. No changes to any object. Uses Get-Label / Get-LabelPolicy. Verify the current unsupported-services list in Microsoft docs. > [CmdletBinding()] param([string]$CsvPath = '')

IPDKE-Scope-And-ServiceHealth.ps1

Purpose: Two safe helpers for a Double Key Encryption rollout: (1) create/update the Microsoft Entra security groups the DKE label is piloted and scoped to — 'Purview - DKE Pilot Users' and 'SG - DKE Ultra Confidential Readers' — and add supplied members; and (2) run a read-only health probe of the customer-held DKE key service endpoint (reachability, HTTPS/TLS, and that it returns public-key JSON). Group changes are gated by -Apply (ShouldProcess/-WhatIf); the endpoint probe is always read-only. HOW TO RUN Install-Module Microsoft.Graph -Scope CurrentUser Probe only (no group change): powershell -ExecutionPolicy Bypass -File .\IPDKE-Scope-And-ServiceHealth.ps1 `-DkeEndpointUrl 'https://dke.contoso.com/DKE-Key-01'` Preview group build: `.\IPDKE-Scope-And-ServiceHealth.ps1 -Members user1@contoso.com,user2@contoso.com -WhatIf` Apply group build + probe: `.\IPDKE-Scope-And-ServiceHealth.ps1 -Members user1@contoso.com -Apply` -DkeEndpointUrl 'https://dke.contoso.com/DKE-Key-01' Sign in with rights to create groups (Groups Administrator) when prompted. EXPECTED RESULTS The named group(s) exist (created if missing, with -Apply) and members are added; the endpoint probe reports PASS/FAIL for HTTPS reachability, TLS, HTTP 200, and JSON-shaped body. .NOTES White-label — Purview Solutions. No secrets. Group writes

gated by -Apply; endpoint probe is read-only (Invoke-WebRequest GET). Uses Microsoft Graph (New-MgGroup / New-MgGroupMemberByRef). > [CmdletBinding(SupportsShouldProcess=$true)] param([switch]$Apply, [string]$GroupName = 'Purview - DKE Pilot Users', [string]$ReadersGroup = 'SG - DKE Ultra Confidential Readers', [string[]]$Members = @(), [string]$DkeEndpointUrl = '')

How to run: Install-Module Microsoft.Graph -Scope CurrentUser Probe only (no group change): powershell -ExecutionPolicy Bypass -File .\IPDKE-Scope-And-ServiceHealth.ps1 `-DkeEndpointUrl 'https://dke.contoso.com/DKE-Key-01' Preview group build: .\IPDKE-Scope-And-ServiceHealth.ps1 -Members user1@contoso.com,user2@contoso.com -WhatIf Apply group build + probe: .\IPDKE-Scope-And-ServiceHealth.ps1 -Members user1@contoso.com -Apply` -DkeEndpointUrl 'https://dke.contoso.com/DKE-Key-01' Sign in with rights to create groups (Groups Administrator) when prompted. EXPECTED RESULTS The named group(s) exist (created if missing, with -Apply) and members are added; the endpoint probe reports PASS/FAIL for HTTPS reachability, TLS, HTTP 200, and JSON-shaped body. .NOTES White-label — Purview Solutions. No secrets. Group writes gated by -Apply; endpoint probe is read-only (Invoke-WebRequest GET). Uses Microsoft Graph (New-MgGroup / New-MgGroupMemberByRef). > [CmdletBinding(SupportsShouldProcess=$true)] param([switch]$Apply, [string]$GroupName = 'Purview - DKE Pilot Users', [string]$ReadersGroup = 'SG - DKE Ultra Confidential Readers', [string[]]$Members = @(), [string]$DkeEndpointUrl = '')

IPDKE-Validation.ps1

Purpose: Checks that the expected named objects for a Double Key Encryption rollout are in place: the DKE sensitivity label exists with encryption enabled and the expected endpoint URL, the label policy is published to the pilot group, and (optionally) the DKE endpoint responds over HTTPS. Use it after Implementation Steps 6–7 and after each expansion. It changes nothing. HOW TO RUN Install-Module ExchangeOnlineManagement -Scope CurrentUser powershell -ExecutionPolicy Bypass -File .\IPDKE-Validation.ps1 `-LabelName 'Sensitivity Label - Ultra Confidential (DKE) - Named Group' -PolicyName 'Label Policy - DKE Ultra Confidential - DKE Pilot' ` -DkeEndpointUrl 'https://dke.contoso.com/DKE-Key-01' Sign in as a Compliance Administrator (or Information Protection Admin) when prompted. EXPECTED RESULTS PASS/FAIL lines for: label exists, encryption enabled, DKE endpoint URL matches, policy exists and is enabled, and (if -DkeEndpointUrl given) the endpoint returns HTTP 200 over HTTPS. .NOTES White-label — Purview Solutions. Read-only. No secrets. No changes to any object. Uses Get-Label / Get-LabelPolicy and a read-only Invoke-WebRequest GET for endpoint reachability. > [CmdletBinding()] param([string]$LabelName = 'Sensitivity Label - Ultra Confidential (DKE) - Named Group', [string]$PolicyName = 'Label Policy - DKE Ultra Confidential - DKE Pilot', [string]$PilotGroupName = 'Purview - DKE Pilot Users', [string]$DkeEndpointUrl = '')

How to run: Install-Module ExchangeOnlineManagement -Scope CurrentUser powershell -ExecutionPolicy Bypass -File .\IPDKE-Validation.ps1 `-LabelName 'Sensitivity Label - Ultra Confidential (DKE) - Named Group' -PolicyName 'Label Policy - DKE Ultra Confidential - DKE Pilot' ` -DkeEndpointUrl 'https://dke.contoso.com/DKE-Key-01' Sign in as a Compliance Administrator (or Information Protection Admin) when prompted. EXPECTED RESULTS PASS/FAIL lines for: label exists, encryption enabled, DKE endpoint URL matches, policy exists and is enabled, and (if -DkeEndpointUrl given) the endpoint returns HTTP 200 over HTTPS. .NOTES White-label — Purview Solutions. Read-only. No secrets. No changes to any object. Uses Get-Label / Get-LabelPolicy and a read-only Invoke-WebRequest GET for endpoint reachability. > [CmdletBinding()] param([string]$LabelName = 'Sensitivity Label - Ultra Confidential (DKE) - Named Group', [string]$PolicyName = 'Label Policy - DKE Ultra Confidential - DKE Pilot', [string]$PilotGroupName = 'Purview - DKE Pilot Users',

```
[string]$DkeEndpointUrl = '' )
```